

Week 4: Introduction to Deep Reinforcement Learning

From tabular updates to neural-network policies and critics

FYS4411 – Reinforcement Learning and Agentic AI
University of Oslo, Department of Physics

June 2026

Learning goals for Week 4

By the end of the lecture, you should be able to:

- ▶ explain why tabular RL breaks in realistic scientific and control problems;
- ▶ formulate value functions and policies as parametrized neural-network functions;
- ▶ derive the basic gradient and semi-gradient updates used in deep RL;
- ▶ describe the core ideas behind DQN, policy gradients, actor-critic methods, and PPO;
- ▶ identify the state, action, reward, network output, and learning signal in deep-RL applications;
- ▶ connect the theory to quantum engineering, molecular dynamics, and robotics.

1. Why deep RL: when tables no longer work.
2. Function approximation: values and policies as neural networks.
3. Losses and updates: SGD, Monte Carlo targets, TD semi-gradients.
4. Deep Q-learning: DQN, target networks, and replay buffers.
5. Policy parametrization: stochastic and continuous policies.
6. Policy gradients, advantages, actor-critic methods, and PPO.
7. Week 4 notebook examples and physical applications.

Why deep reinforcement learning?

Tabular methods assume that we can store one number for each state or state-action pair:

$$V(s), \quad Q(s, a).$$

This is helpful for learning the formalism, but unrealistic when the state contains:

- ▶ a quantum wavefunction, density matrix, or measurement history;
- ▶ molecular coordinates, velocities, energies, or collective variables;
- ▶ robot joint angles, velocities, contacts, force readings, and camera images.

Takeaway

Deep RL replaces explicit tables by trainable functions, allowing experience from one region of state space to generalize to nearby or structurally similar regions.

The old object: a table

In tabular value-based RL, the action-value function is represented explicitly:

$$Q : \mathcal{S} \times \mathcal{A} \rightarrow \mathbb{R}.$$

For a finite action set, this is just a table.

state	$Q(s, a_1)$	$Q(s, a_2)$	$Q(s, a_3)$
s_1	0.2	0.5	0.1
s_2	0.7	0.3	0.4
s_3	0.1	0.2	0.9

A tabular update changes one entry, for example $Q(S_t, A_t)$.

The new object: a parametrized function

Deep RL replaces the table by a parametrized function:

$$q_{\pi}(s, a) \approx \hat{q}(s, a; w),$$

where w are neural-network parameters.



Changing w changes many value estimates at once. This is both the power and the danger of function approximation.

Why this is not just supervised learning

In supervised learning, the data are usually fixed:

$$\mathcal{D} = \{(x_i, y_i)\}_{i=1}^N, \quad \theta^* \in \arg \min_{\theta} L(\theta).$$

The model can change its predictions, but it does not change the dataset. In RL, the policy changes the future data:

$$S_t \xrightarrow{\pi_{\theta}} A_t \longrightarrow R_{t+1}, S_{t+1}.$$

Takeaway

Deep RL uses neural-network optimization, but the data and targets are generated by interaction with the environment and may change as the policy changes.

Three neural-network objects

Object	Network form	Meaning
state value	$v_{\pi}(s) \approx \hat{v}(s; w)$	how good is state s ?
action value	$q_{\pi}(s, a) \approx \hat{q}(s, a; w)$	how good is action a in state s ?
policy	$\pi(a s) \approx \pi_{\theta}(a s)$	how does the agent choose actions?

These lead to three major families: value-based methods, policy-based methods, and actor-critic methods.

The common update pattern

Many tabular RL updates have the form

$$\text{estimate} \leftarrow \text{estimate} + \alpha[\text{target} - \text{estimate}].$$

With a neural network, the estimate is a function output. The same idea becomes gradient descent on a loss:

$$L(w) = \frac{1}{2}[\text{target} - f_w(\text{input})]^2.$$

The update is

$$w \leftarrow w - \alpha \nabla_w L(w).$$

Takeaway

The RL algorithm supplies the target; backpropagation supplies the parameter update.

Warm-up: ordinary SGD

For fixed target y and network prediction $f(x; w)$, use

$$L(w) = \frac{1}{2}[y - f(x; w)]^2.$$

Then

$$\nabla_w L(w) = -[y - f(x; w)]\nabla_w f(x; w),$$

so gradient descent gives

$$w \leftarrow w + \alpha[y - f(x; w)]\nabla_w f(x; w).$$

This is a useful template, but in RL the target is usually not a fixed label.

Monte Carlo value approximation

For policy evaluation,

$$v_{\pi}(s) = \mathbb{E}_{\pi}[G_t \mid S_t = s].$$

A value network represents

$$v_{\pi}(s) \approx \hat{v}(s; w).$$

After an episode, the sampled return G_t becomes a target:

$$L_t(w) = \frac{1}{2}[G_t - \hat{v}(S_t; w)]^2.$$

The update is

$$w \leftarrow w + \alpha[G_t - \hat{v}(S_t; w)]\nabla_w \hat{v}(S_t; w).$$

Takeaway

Monte Carlo function approximation fits a neural value function to complete sampled returns.

Temporal-difference semi-gradient update

TD learning replaces the complete return by a bootstrapped target:

$$Y_t = R_{t+1} + \gamma \hat{v}(S_{t+1}; w).$$

The TD error is

$$\delta_t = R_{t+1} + \gamma \hat{v}(S_{t+1}; w) - \hat{v}(S_t; w).$$

The standard semi-gradient update is

$$w \leftarrow w + \alpha \delta_t \nabla_w \hat{v}(S_t; w).$$

This moves the current prediction toward the one-step target.

What is “semi” about?

The squared TD error is

$$\frac{1}{2} [R_{t+1} + \gamma \hat{v}(S_{t+1}; w) - \hat{v}(S_t; w)]^2.$$

The target contains w through $\hat{v}(S_{t+1}; w)$. In semi-gradient TD, we treat the target as fixed during the update:

$$\nabla_w \left(\frac{1}{2} [Y_t - \hat{v}(S_t; w)]^2 \right) = -[Y_t - \hat{v}(S_t; w)] \nabla_w \hat{v}(S_t; w).$$

Takeaway

The gradient is taken through the current prediction, not through the bootstrapped target.

From tabular Q-learning to neural Q-learning

Tabular Q-learning:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha \left[R_{t+1} + \gamma \max_{a'} Q(S_{t+1}, a') - Q(S_t, A_t) \right].$$

Deep Q-learning replaces

$$Q(s, a) \quad \text{by} \quad \hat{q}(s, a; w).$$

The target becomes

$$Y_t = R_{t+1} + \gamma \max_{a'} \hat{q}(S_{t+1}, a'; w),$$

and the loss is

$$L_t(w) = \frac{1}{2} [Y_t - \hat{q}(S_t, A_t; w)]^2.$$

Deep Q-network output

For discrete actions, a DQN often maps one state to many action values:

$$f_w(s) = (\hat{q}(s, a_1; w), \dots, \hat{q}(s, a_m; w)).$$

The greedy action is

$$a^* \in \arg \max_a \hat{q}(s, a; w).$$



DQN as sample-based Bellman fitting

Given a transition sample

$$(S_t, A_t, R_{t+1}, S_{t+1}),$$

DQN fits the current prediction to a Bellman target:

$$\hat{q}(S_t, A_t; w) \approx R_{t+1} + \gamma \max_{a'} \hat{q}(S_{t+1}, a'; w^-).$$

Here w^- may denote a target network.

Interpretation

DQN is still Q-learning. The table has simply been replaced by a neural network and the table update has become a gradient update.

Why target networks?

Without a target network, the target

$$R_{t+1} + \gamma \max_{a'} \hat{q}(S_{t+1}, a'; w)$$

uses the same parameters being updated. This creates a moving-target problem: the network chases labels that change whenever the network changes. DQN stabilizes learning by using separate target parameters:

$$Y_t = R_{t+1} + \gamma \max_{a'} \hat{q}(S_{t+1}, a'; w^-).$$

The online network w is updated frequently; w^- is updated more slowly.

Why experience replay?

Consecutive transitions are highly correlated:

$$(S_t, A_t, R_{t+1}, S_{t+1}), \quad (S_{t+1}, A_{t+1}, R_{t+2}, S_{t+2}).$$

Training on only the most recent transitions can make optimization unstable.
Experience replay stores transitions in a buffer

$$\mathcal{D} = \{(s_i, a_i, r_i, s'_i)\}_{i=1}^N$$

and samples mini-batches for gradient updates.

Takeaway

Replay reuses experience and partially breaks the temporal correlation in the training data.

DQN mini-batch loss

For a sampled mini-batch B , the DQN loss can be written

$$L(w) = \frac{1}{|B|} \sum_{(s,a,r,s') \in B} \frac{1}{2} [y(r, s') - \hat{q}(s, a; w)]^2,$$

where

$$y(r, s') = r + \gamma \max_{a'} \hat{q}(s', a'; w^-).$$

If s' is terminal, omit the continuation term. This is a supervised-looking loss, but the labels are Bellman targets generated from sampled transitions.

DQN: strengths and limits

Strengths

- ▶ Works with large state spaces.
- ▶ Good for discrete actions.
- ▶ Learns values and derives a policy.
- ▶ Can use replay buffers efficiently.

Limits

- ▶ Less natural for continuous actions.
- ▶ Can be unstable with function approximation.
- ▶ Needs exploration strategy.
- ▶ Max operator can amplify value errors.

Reference: Mnih et al., Nature 2015; DQN introduced target networks and replay in deep value learning.

Policy parametrization

Value-based methods learn values and derive policies. Policy-based methods represent the policy directly:

$$\pi_{\theta}(a \mid s) = \mathbb{P}_{\theta}(A_t = a \mid S_t = s).$$

For a finite action space, a neural network outputs logits

$$z_{\theta}(s, a_1), \dots, z_{\theta}(s, a_m),$$

which are converted to probabilities by softmax:

$$\pi_{\theta}(a_i \mid s) = \frac{\exp(z_{\theta}(s, a_i))}{\sum_{j=1}^m \exp(z_{\theta}(s, a_j))}.$$

Continuous policies

For continuous control, the action may be a real-valued vector:

$$a_t \in \mathbb{R}^d.$$

A policy network can output distribution parameters, for example

$$\pi_{\theta}(a \mid s) = \mathcal{N}(\mu_{\theta}(s), \Sigma_{\theta}(s)).$$

During training, actions are sampled:

$$A_t \sim \pi_{\theta}(\cdot \mid S_t).$$

At evaluation, one may use the mean action $\mu_{\theta}(S_t)$.

Takeaway

Policy parametrization is especially natural when actions are continuous: torques, forces, voltages, phases, amplitudes, and steering commands.

The policy objective

The objective is expected return:

$$J(\theta) = \mathbb{E}_{\pi_{\theta}}[G_0].$$

Equivalently, in an episodic setting,

$$J(\theta) = \mathbb{E}_{\pi_{\theta}} \left[\sum_{t=0}^{T-1} \gamma^t R_{t+1} \right].$$

The optimization problem is

$$\theta^* \in \arg \max_{\theta} J(\theta).$$

The challenge is that changing θ changes the actions, which changes the distribution of future states and trajectories.

Trajectory probabilities

For a finite trajectory

$$\tau = (s_0, a_0, r_1, s_1, \dots, s_T),$$

the probability under policy π_θ is

$$P_{\pi_\theta}(\tau) = \mu_0(s_0) \prod_{t=0}^{T-1} \pi_\theta(a_t \mid s_t) p(s_{t+1}, r_{t+1} \mid s_t, a_t).$$

Only the policy factors depend on θ .

Takeaway

Policy-gradient methods can be model-free because we do not need to differentiate through the environment transition probabilities.

The log-derivative trick

The identity

$$\nabla_{\theta} P_{\pi_{\theta}}(\tau) = P_{\pi_{\theta}}(\tau) \nabla_{\theta} \log P_{\pi_{\theta}}(\tau)$$

turns a gradient of probabilities into an expectation. Since only the policy depends on θ ,

$$\nabla_{\theta} \log P_{\pi_{\theta}}(\tau) = \sum_{t=0}^{T-1} \nabla_{\theta} \log \pi_{\theta}(A_t | S_t).$$

This gives the basic policy-gradient form.

Policy-gradient update

A Monte Carlo policy-gradient estimator is

$$\nabla_{\theta} J(\theta) \approx \sum_{t=0}^{T-1} \nabla_{\theta} \log \pi_{\theta}(A_t | S_t) G_t.$$

The corresponding ascent update is

$$\theta \leftarrow \theta + \alpha \sum_{t=0}^{T-1} \nabla_{\theta} \log \pi_{\theta}(A_t | S_t) G_t.$$

Interpretation

Increase the probability of actions that were followed by high return; decrease the probability of actions that were followed by low return.

Why use a baseline?

Policy-gradient estimates can have high variance. A baseline reduces variance:

$$\theta \leftarrow \theta + \alpha \sum_{t=0}^{T-1} \nabla_{\theta} \log \pi_{\theta}(A_t | S_t) [G_t - b(S_t)].$$

A natural baseline is a value estimate:

$$b(S_t) \approx v_{\pi_{\theta}}(S_t).$$

Then

$$G_t - b(S_t)$$

measures whether the outcome was better or worse than expected from that state.

Takeaway

This idea leads directly to advantages and actor-critic methods.

The advantage function

The advantage of action a in state s is

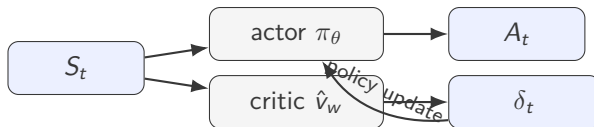
$$A_{\pi}(s, a) = q_{\pi}(s, a) - v_{\pi}(s).$$

It answers:

Was this action better or worse than the typical action chosen by the policy in this state?

If $A_{\pi}(s, a) > 0$, increase the probability of the action. If $A_{\pi}(s, a) < 0$, decrease it. In practice, the advantage must be estimated from data.

Actor-critic: the basic architecture



The actor chooses actions; the critic evaluates the state or action and supplies a learning signal.

Actor-critic TD error

A simple critic is a state-value network

$$\hat{v}(S_t; w).$$

The critic computes the TD error

$$\delta_t = R_{t+1} + \gamma \hat{v}(S_{t+1}; w) - \hat{v}(S_t; w).$$

This can be interpreted as a one-step advantage estimate. If $\delta_t > 0$, the action produced a better-than-expected transition. If $\delta_t < 0$, it was worse than expected.

Actor and critic updates

Critic update:

$$w \leftarrow w + \alpha_w \delta_t \nabla_w \hat{v}(S_t; w).$$

Actor update:

$$\theta \leftarrow \theta + \alpha_\theta \delta_t \nabla_\theta \log \pi_\theta(A_t | S_t).$$

Critic

- ▶ learns values;
- ▶ uses TD learning;
- ▶ reduces variance.

Actor

- ▶ learns policy;
- ▶ changes action probabilities;
- ▶ uses critic feedback.

Why actor-critic matters

Actor-critic methods combine the main advantages of two worlds:

- ▶ value learning gives a local estimate of long-term consequences;
- ▶ policy learning directly optimizes behavior;
- ▶ TD learning makes updates possible after individual transitions;
- ▶ stochastic policies allow exploration and continuous-action control.

Takeaway

Many modern deep-RL algorithms are actor-critic methods with additional stabilization mechanisms.

PPO: why control the policy update?

Policy-gradient updates can be unstable if the new policy becomes too different from the policy that generated the data. Let $\pi_{\theta_{\text{old}}}$ generate the sampled actions. PPO compares the new and old probabilities using

$$r_t(\theta) = \frac{\pi_{\theta}(A_t | S_t)}{\pi_{\theta_{\text{old}}}(A_t | S_t)}.$$

If $r_t(\theta)$ is too far from 1, the policy has changed a lot for the sampled action.

PPO clipped objective

PPO uses a clipped surrogate objective:

$$L^{\text{clip}}(\theta) = \mathbb{E}_t \left[\min \left(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t \right) \right].$$

The clipping limits the benefit of moving the new policy too far from the old policy.

Takeaway

PPO keeps the policy-gradient idea, but makes updates more conservative and easier to tune in practice.

Reference: Schulman et al., PPO, arXiv:1707.06347.

Where the algorithms sit

Method	Network learns	Typical use
DQN	$\hat{q}(s, a; w)$	large states, discrete actions
Policy gradient	$\pi_{\theta}(a s)$	direct stochastic policy optimization
Actor-critic	π_{θ} and $\hat{v}_w$ or $\hat{q}_w$	continuous control, lower-variance policy learning
PPO	actor and critic with clipped update	robust on-policy deep RL
DDPG/SAC-style methods	actor and critic	continuous actions, off-policy training

On-policy and off-policy in deep RL

On-policy

- ▶ data generated by current policy;
- ▶ learn/evaluate that same policy;
- ▶ examples: vanilla policy gradient, PPO.

Off-policy

- ▶ data may come from another behavior policy or replay buffer;
- ▶ learn a target policy or value function;
- ▶ examples: DQN, DDPG, many replay-based methods.

Deep RL often trades off data efficiency, stability, and how safely the agent can explore.

Practical issues in deep RL

Replacing tables by neural networks introduces several issues:

- ▶ **moving targets**: the target may depend on current networks;
- ▶ **correlated data**: consecutive transitions are not i.i.d.;
- ▶ **exploration**: the agent must collect useful experience;
- ▶ **variance**: policy-gradient estimates can be noisy;
- ▶ **bias**: TD targets bootstrap from learned estimates;
- ▶ **large updates**: a policy can change too much in one step.

Takeaway

Modern deep RL is often about stabilizing the basic Bellman and policy-gradient ideas under nonlinear function approximation.

Week 4 notebook: LineWorld as a microscope

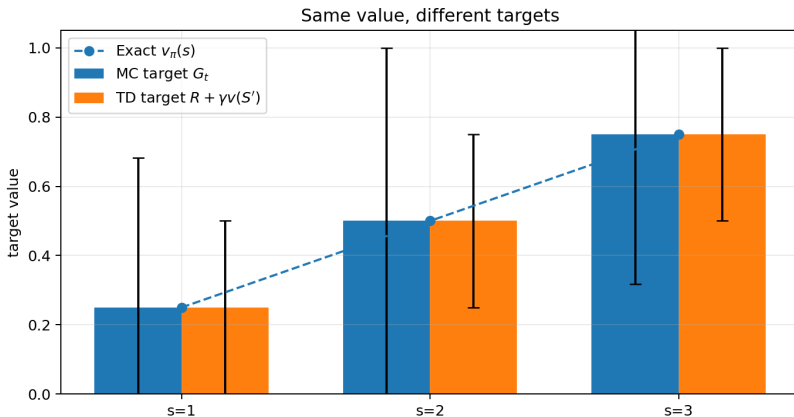
The notebook uses a small LineWorld environment:

0 1 2 3 4,

with terminal states 0 and 4, reward 1 at the right terminal, and actions left/right. Why use a tiny problem in a deep-RL lecture?

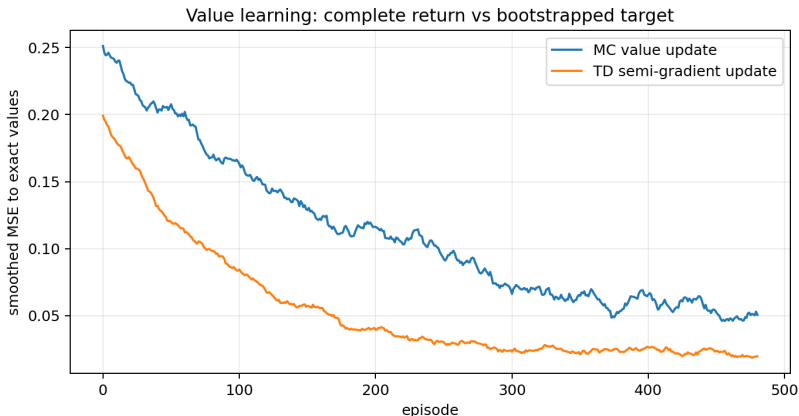
- ▶ the exact value function is known;
- ▶ MC and TD targets can be compared directly;
- ▶ actor and critic updates can be inspected visually;
- ▶ the same neural-network machinery appears in a setting that is not a black box.

Notebook figure: MC target versus TD target



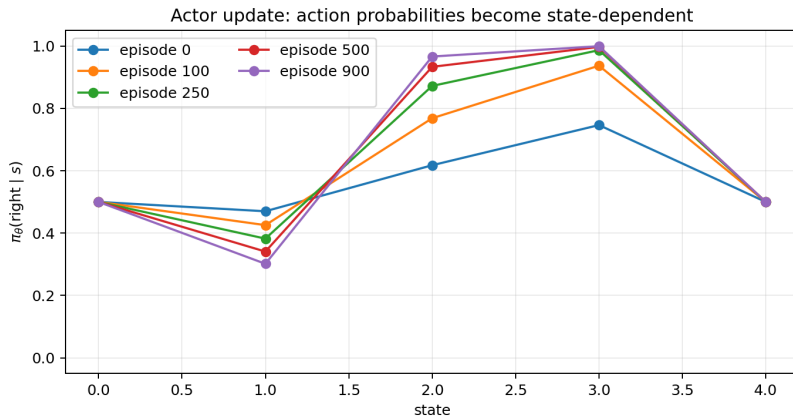
Monte Carlo fits complete returns G_t . TD fits one-step targets $R_{t+1} + \gamma \hat{v}(S_{t+1}; w)$.

Notebook figure: learning curves



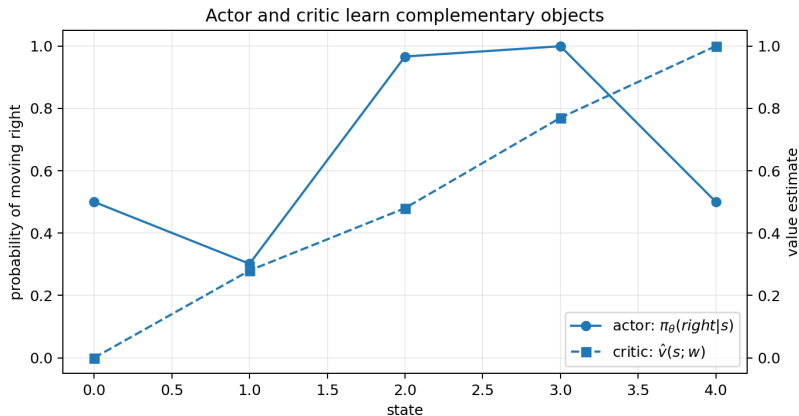
The point is not that TD is always better. The point is that TD learns from shorter, bootstrapped targets, while MC waits for complete returns.

Notebook figure: actor policy evolution



The actor changes a distribution over actions. Here the learned policy becomes more likely to move toward the rewarding terminal state.

Notebook figure: actor and critic together



The actor chooses actions; the critic estimates values. Actor-critic learning couples the two updates.

What exactly was updated?

Notebook part	Network output	Learning signal
MC value learning	$\hat{v}(s; w)$	complete return G_t
TD value learning	$\hat{v}(s; w)$	TD error δ_t
Actor	$\pi_\theta(a \mid s)$	$\delta_t \nabla_\theta \log \pi_\theta$
Critic	$\hat{v}(s; w)$	semi-gradient TD

Takeaway

The same small environment demonstrates the central transition from table updates to gradient updates.

Application map for FYS4411

Domain	Why tabular fails	Natural deep-RL object
Quantum engineering	wavefunctions, density matrices, histories	pulse policy or value network
Molecular dynamics	high-dimensional configurations, rare events	biasing policy or sampling controller
Robotics	continuous states/actions, contacts, sensors	torque/velocity policy with critic

The shared structure is sequential control under uncertainty and delayed consequences.

Quantum engineering: controlled dynamics

A controlled quantum system can be written

$$H(t) = H_0 + \sum_k u_k(t) H_k.$$

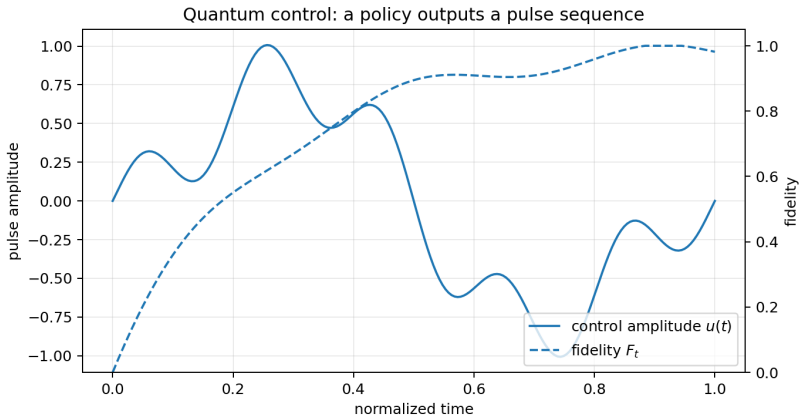
The agent chooses controls such as amplitudes, phases, detunings, or durations:

$$A_t = (u_1(t), \dots, u_K(t)).$$

The state may be a wavefunction, density matrix, Bloch vector, measurement record, or reduced observable. A common objective is final fidelity:

$$R_T = |\langle \psi_{\text{target}} | \psi_T \rangle|^2 - \lambda \sum_t \|u_t\|^2.$$

Quantum pulse control as deep RL



The policy maps the current observation and time to a control amplitude. The reward may only arrive after the full pulse sequence.

Why deep RL in quantum control?

- ▶ The state may be continuous or high-dimensional.
- ▶ Measurements are stochastic and may disturb the system.
- ▶ Rewards such as fidelity can be delayed until the end of a protocol.
- ▶ Control amplitudes and phases are often continuous.
- ▶ A learned policy can adapt to noise, disorder, or partial observations.

Algorithmic match

DQN fits discrete pulse choices; actor-critic and PPO fit continuous pulse amplitudes or feedback policies.

Quantum examples to mention

- ▶ **State preparation**: choose pulses that prepare a target state.
- ▶ **Gate synthesis**: optimize controls for high-fidelity quantum gates.
- ▶ **Feedback control**: use measurements to choose the next control.
- ▶ **Error correction**: learn syndrome-response strategies.
- ▶ **Metrology**: adapt measurement settings to reduce uncertainty.

Reference: Bukov and Marquardt, Reinforcement Learning for Quantum Technology, arXiv:2601.18953.

Quantum formulation: what is the RL tuple?

RL object	Quantum-control interpretation
state/observation	$ \psi_t\rangle$, ρ_t , Bloch vector, measurement history, time step
action	pulse amplitude, phase, detuning, measurement basis, gate choice
reward	final fidelity, robustness, sensitivity, energy/time penalty
policy	pulse selector or feedback controller
value/critic	long-term expected fidelity/cost from current control state

Molecular dynamics: rare events

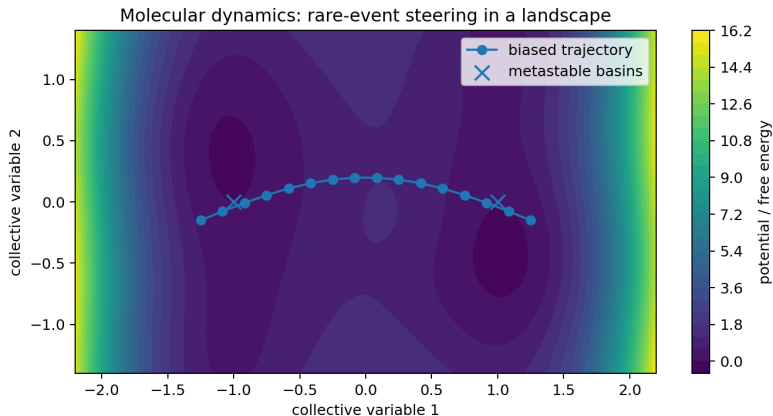
Molecular systems often evolve in high-dimensional configuration spaces and may spend long times in metastable basins. A simplified controlled dynamics can be written

$$X_{t+1} = X_t - \nabla U(X_t)\Delta t + A_t\Delta t + \sqrt{2D\Delta t}\xi_t.$$

Here A_t may represent a learned biasing force or bias-potential parameter. The reward might encourage transition discovery while penalizing unphysical forcing:

$$R_{t+1} = \text{progress toward target basin} - \lambda\|A_t\|^2.$$

MD application: enhanced sampling



Deep RL becomes relevant when the bias depends on many collective variables or when fixed heuristics do not explore efficiently.

MD: what is the policy?

The policy need not act on every atom. It may act on collective variables:

$$s_t = (\xi_1(X_t), \dots, \xi_m(X_t), U_t, T_t, t).$$

Possible actions include:

- ▶ choose a biasing force along a collective variable;
- ▶ choose a window or umbrella-sampling center;
- ▶ choose a temperature or replica-exchange proposal;
- ▶ choose where to restart or branch new simulations.

Takeaway

In MD, the scientific state representation is often as important as the RL algorithm.

MD: why not just run longer?

- ▶ Relevant transitions may be rare compared with simulation time scales.
- ▶ The system may repeatedly revisit already explored basins.
- ▶ Good collective variables may be unknown or incomplete.
- ▶ Simulations can be expensive, so adaptive sampling matters.

Deep-RL role

A neural policy or value function can learn where additional simulation effort, bias, or control is most useful.

Reference: Zhang, Wang, and E, Reinforced dynamics for enhanced sampling, arXiv:1712.03461.

MD caveat: RL extension, not basic MD

The equations of motion, force field, and integrator are still physics. RL enters when we introduce a decision problem:

- ▶ which bias should be applied now?
- ▶ which trajectory should be extended?
- ▶ which temperature or protocol should be selected?
- ▶ which region of configuration space should be sampled next?

Takeaway

RL does not replace molecular dynamics; it can control or allocate simulation effort around the dynamics.

Robotics: continuous feedback control

A robot policy maps sensor state to continuous control:

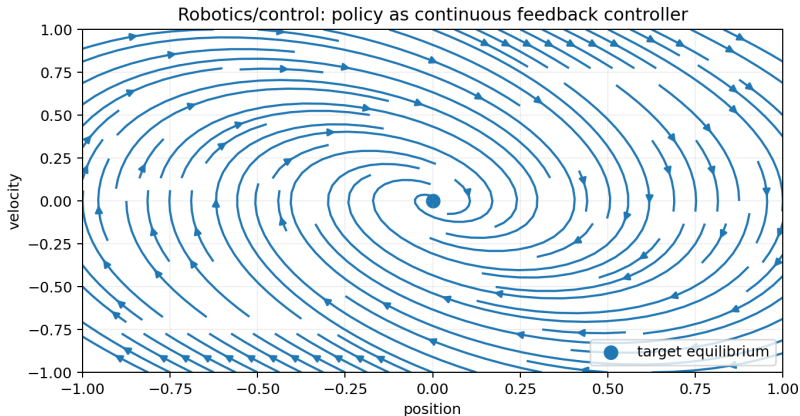
$$\pi_{\theta} : (q_t, \dot{q}_t, \text{sensors}) \mapsto \tau_t.$$

Here q_t are joint angles and τ_t are torques. A typical reward combines progress, stability, and penalties:

$$R_t = \text{task progress} - \lambda_1 \text{energy} - \lambda_2 \text{fall/collision penalty}.$$

Deep RL is natural because both state and action spaces are continuous and high-dimensional.

Robotics/control as policy learning



The policy is a feedback controller. The critic estimates whether the current state-action choice improves long-term behavior.

- ▶ **Locomotion**: walking, hopping, balancing, recovery from pushes.
- ▶ **Manipulation**: grasping, pushing, insertion, tool use.
- ▶ **Navigation**: steering under noise, obstacles, and partial observations.
- ▶ **Assistive devices**: prosthetics, exoskeletons, rehabilitation controllers.

Continuous-control algorithms such as deterministic actor-critic methods and PPO-style methods are common starting points.

Reference: Lillicrap et al., Continuous control with deep RL, arXiv:1509.02971.

Robotics caveat: sim-to-real

Training often happens in simulation because real-world exploration can be unsafe or expensive. The difficulty is the reality gap:

- ▶ friction, delays, contacts, and compliance may differ;
- ▶ sensors may be noisy or biased;
- ▶ rare failures may not appear in training;
- ▶ a high reward in simulation does not guarantee safe deployment.

Takeaway

For physical systems, deep RL should be coupled with constraints, domain randomization, verification, and human oversight.

Choosing the algorithm by action space

Action type	Good starting point	Example
small discrete	DQN / deep SARSA	bang-bang quantum pulses, grid navigation
large discrete	policy gradients / actor-critic	circuit construction, tool choice
continuous	actor-critic, PPO, DDPG/SAC-style	torques, pulse amplitudes, biasing forces
hybrid	hierarchical or structured policies	experiments with discrete mode and continuous controls

Choosing the algorithm by reward structure

Reward pattern	Challenge	Useful idea
terminal fidelity	delayed credit assignment	MC targets, reward shaping, actor-critic
stepwise progress	easier online learning	TD learning, advantage estimates
rare success	sparse data	exploration bonuses, curriculum, offline data
safety penalties	avoid bad exploration	constraints, safe RL, simulation

A physics-first DRL workflow

1. Define the physical state or observation: what is actually available?
2. Define admissible actions: what can the controller change?
3. Define reward carefully: what scientific objective is being optimized?
4. Choose a representation: table, features, neural network, or structured model.
5. Choose an algorithm: value-based, policy-based, or actor-critic.
6. Compare with baselines: hand-designed controls, optimal control, random policy.
7. Stress-test robustness: noise, parameter shifts, constraints, unseen initial states.

Discussion questions

Discussion

In a quantum-control experiment, should the state be the full wavefunction, a density matrix, a measurement record, or experimentally accessible observables?

Discussion

In molecular dynamics, is the reward based on physical free-energy sampling, fast barrier crossing, or something else?

Discussion

In robotics, when is it acceptable for the agent to explore, and when must exploration be constrained or simulated?

What to remember

- ▶ Deep RL is not a new objective: the goal is still large expected return.
- ▶ The main change is representation:

$$V(s) \rightarrow \hat{v}(s; w), \quad Q(s, a) \rightarrow \hat{q}(s, a; w), \quad \pi(a | s) \rightarrow \pi_{\theta}(a | s).$$

- ▶ Value methods fit Bellman targets; policy methods update action probabilities.
- ▶ Actor-critic methods use a critic to produce lower-variance policy updates.
- ▶ Applications require careful state design, reward design, constraints, and baselines.

References and useful links

- ▶ FYS4411 notes, Chapter 3: Function Approximation and Deep Reinforcement Learning.
- ▶ Mnih et al. “Human-level control through deep reinforcement learning.” Nature 2015.
- ▶ Schulman et al. “Proximal Policy Optimization Algorithms.” arXiv:1707.06347.
- ▶ Lillicrap et al. “Continuous control with deep reinforcement learning.” arXiv:1509.02971.
- ▶ Bukov and Marquardt. “Reinforcement Learning for Quantum Technology.” arXiv:2601.18953.
- ▶ Zhang, Wang, and E. “Reinforced dynamics for enhanced sampling in large atomic and molecular systems.” arXiv:1712.03461.